第3回：Webアプリへの組み込み

ブラウザからFastAPIを経由してLLMを呼ぶ

今日のゴールと流れ

質問がブラウザからLLMまで、どのファイルを通るかを追いかけられる。
サーバーが入力を確かめる理由と、止まる場所の違いを説明できる。

内容
通信の2段階、質問が通るファイル
履歴の置き場所、JSON、Pydantic
完成版 `chat-app/` を動かす・読む
穴埋め：app.js の送信、main.py の確認
`/docs` で 422 と 400 を確認、提出

API利用時は毎回、起動するターミナルで：
`export OPENAI_API_KEY=sk-********`
終了時は停止して `unset OPENAI_API_KEY` 。

今日の進め方

段階	すること
1	`sample-app/chat-app/` の完成版を動かす
2	コードの説明を聞く
3	`session03/exercise/` の穴あきコードを埋めて動かす
4	コミットしてプッシュする
5	GitHubのURLをフォームで提出する

穴あきコードの答えは `sample-app/chat-app/` です。

通信の2段階

1. ブラウザが自分のFastAPIに履歴を送る
2. FastAPIが外部のLLM APIに履歴を送る
3. LLMの結果をFastAPIが受け取る
4. FastAPIがJSONを返し、ブラウザが表示する

通信	送る側	受け取る側	APIキー
1	ブラウザ	自分のFastAPI	使わない
2	自分のPython	外部のLLMサービス	使う

ブラウザへ送るのは回答です。APIキーはサーバー側だけにあります。

質問が通るファイル

順番	実行場所	ファイルと仕事
1	ブラウザ	`app.js` が履歴をJSONにして送る
2	サーバー	`main.py` の `chat()` が履歴を確かめる
3	サーバー	`llm.py` の `ask_with_history()` がLLM APIを呼ぶ
4	サーバー	`main.py` が結果の辞書を返す
5	ブラウザ	`app.js` が回答を履歴に足して表示する

`ask_with_history(history, model, instructions)` は前回の `chat.py` と同じ考え方です。

今回のファイル

ファイル	役割
`main.py`	自作API `/api/chat` と、画面の配信
`llm.py`	OpenAIのAPIを呼ぶ `ask_with_history()`
`static/index.html`	入力欄、ボタン、会話の一覧
`static/app.js`	履歴をためる、送信する、表示する
`static/style.css`	見た目

読み順は `static/app.js` 、 `main.py` の `chat()` 、 `llm.py` です。
前回の `llm-call/chat.py` の `input()` が、Web画面に置き換わります。

履歴はどこにあるか

履歴はブラウザの `history` 変数だけが持ちます。サーバーは保存しません。

```
// 会話の履歴。質問と回答を、この順番でためていく。
let history = [];
```

性質	理由
何人で使っても会話が混ざらない	履歴は各ブラウザの中にある
再読み込みで履歴が消える	変数が作り直される
サーバーは送られた履歴を確かめる	書き換えられている可能性がある

完成版を動かす

```
source .venv/bin/activate
cd sample-app/chat-app
python main.py
```

ブラウザでポート8000番を開きます。

`main.py` は、画面のファイルとAPIの両方を配信します。

試すこと	期待する結果
「私の名前は佐藤です。」の次に「私の名前は？」	答えられる
「会話をリセット」の後に「私の名前は？」	答えられない
質問を重ねる	入力トークン数が増える

送るJSONの形

ブラウザからFastAPIへ送る本文です。これまでの会話を全部入れます。

```
{"history": [
  {"role": "user", "content": "私の名前は佐藤です。"},
  {"role": "assistant", "content": "こんにちは、佐藤さん。"},
  {"role": "user", "content": "私の名前は？"}
]}
```

`role` は `user`（質問）か `assistant`（LLMの回答）です。
最後の1件が、今回の質問です。

同じ履歴の3つの形

場所	表現
JavaScriptの配列	`[{role: 'user', content: '…'}]`
HTTPで送るJSON文字列	`{"history":[{"role":"user","content":"…"}]}`
Pythonの受信後	`request.history` が `Message` のリスト

`JSON.stringify()` は送信用の文字列へ変換します。
FastAPIはJSONを読み取り、Pydanticのモデルで検証します。
履歴の中身は、変換の前後で変わりません。

Pydanticによる入力の形

```
class Message(BaseModel):
    role: Literal['user', 'assistant']
    content: str = Field(min_length=1, max_length=3000)


class ChatRequest(BaseModel):
    history: list[Message] = Field(min_length=1)
```

型・値・長さをサーバーで検証します。

`role` が `system` のデータや、空の `content` は、ここで止まります。

ブラウザの `maxlength` だけでは、別のプログラムからの入力を防げません。

chat() の3つの確認

Pydanticを通った履歴を、さらに `chat()` で確かめます。

```
question = history[-1]
if question['role'] != 'user':
    raise HTTPException(status_code=400, detail='最後は質問（user）にしてください。')
if len(question['content']) > 500:
    raise HTTPException(status_code=400, detail='質問は500文字以内で入力してください。')
if len(history) > MAX_TURNS * 2 - 1:
    raise HTTPException(status_code=400, detail=f'質問は{MAX_TURNS}回までです。会話をリセットしてください。')
```

`history[-1]` はリストの最後の要素です。
形は正しくても、意味として受け付けない履歴を止めます。

422と400の違い

止まる場所	ステータス	例
Pydantic（`Message`、`ChatRequest`）	422	`history` が空
Pydantic	422	`role` が `system`
Pydantic	422	`content` が空
`chat()` の確認	400	最後が `assistant`
`chat()` の確認	400	質問が501文字
`chat()` の確認	400	質問が11回目（履歴21件）

422はFastAPIが自動で返します。400は自分で `raise` します。
どちらでも、LLM APIは呼び出しません。

MAX_TURNS は2か所にある

ファイル	書き方	役割
main.py	MAX_TURNS = 10	サーバーで確かめる
app.js	const MAX_TURNS = 10;	送る前に画面で止める

```
if (history.length / 2 >= MAX_TURNS) {
  statusText.textContent = '質問は' + MAX_TURNS + '回までです。会話をリセットしてください。';
  return;
}
```

画面の確認は使いやすさのため、サーバーの確認は守りのためです。
2か所を同じ値にします。

FastAPIからLLMを呼ぶ

```
try:
    result = ask_with_history(history, MODEL, INSTRUCTIONS)
except ValueError as error:
    raise HTTPException(status_code=400, detail=str(error))
except APITimeoutError:
    raise HTTPException(status_code=504, detail='APIの応答が遅れています。時間をおいて試してください。')
except RateLimitError:
    raise HTTPException(status_code=429, detail='APIの利用制限です。利用枠を確認してください。')
except APIError:
    raise HTTPException(status_code=502, detail='API呼び出しに失敗しました。キー・モデル・利用枠を確認してください。')
return result
```

前回の `except` が、 `print` から `HTTPException` に変わりました。

エラー時のHTTPステータス

例外	ステータス	画面に出る文
`ValueError`	400	回答が完了しませんでした。…
`APITimeoutError`	504	APIの応答が遅れています。…
`RateLimitError`	429	APIの利用制限です。…
`APIError`	502	API呼び出しに失敗しました。…

`detail` の文が、そのまま画面のステータス欄に表示されます。
APIのエラー全文には内部の情報が含まれることがあるので、決まった文だけを返します。

静的ファイルの配信

```
app.mount('/', StaticFiles(directory=BASE_DIR / 'static', html=True), name='static')
```

項目	意味
`'/'`	ルートのURLで画面を配信する
`BASE_DIR / 'static'`	公開するのは `static/` だけ
`html=True`	`/` で `index.html` を返す

APIの定義より後に書きます。前に書くと `/api/chat` も静的ファイル扱いになります。
アプリのフォルダ全体を公開すると、`main.py` や `llm.py` が見えてしまいます。

履歴を画面に描く

```
function showMessages() {
  messagesList.replaceChildren(); // 前の表示を消す
  for (const message of history) {
    const item = document.createElement('li');
    if (message.role === 'user') {
      item.className = 'user';
      item.textContent = 'あなた: ' + message.content;
    } else {
      item.className = 'assistant';
      item.textContent = 'LLM: ' + message.content;
    }
    messagesList.appendChild(item);
  }
}
```

安全な表示

`showMessages()` は、 `history` の内容を毎回まるごと描き直します。
差分を考えないので、履歴と画面がずれません。

書き方	扱い
`textContent`	文字列として表示する
`innerHTML`	HTMLとして解釈する

LLMの出力も、ユーザー入力と同じように扱います。
`innerHTML` へ入れると、出力に含まれるタグやスクリプトが動きます。
教材ではMarkdownの装飾を付けず、文字列として表示します。

sendQuestion() の流れ

```
// 1. 今回の質問を履歴に追加して表示する。
history.push({role: 'user', content: question});
showMessages();
questionInput.value = '';
// 回答を待つ間は、送信もリセットもできないようにする。
sendButton.disabled = true;
resetButton.disabled = true;
statusText.textContent = '回答を待っています…';
```

番号	すること
1	質問を `history` に足す
2	`history` をまとめてサーバーへ送る（穴あき）
3	回答を `history` に足す

fetchによる送信

```
// 2. 履歴をまとめてサーバーに送る。
const response = await fetch('/api/chat', {
  method: 'POST',
  headers: {'Content-Type': 'application/json'},
  body: JSON.stringify({history: history})
});
const data = await response.json();
```

相対パスなので、画面を配信した同じサーバーへ送ります。

`JSON.stringify()` で、配列をJSON文字列にします。

`body` に入るのは今回の質問だけでなく、 `history` の全部です。

asyncとawait

`fetch()` の通信結果は、すぐには返りません。

書き方	意味
`async function`	待ち時間のある処理を書く関数
`await`	その処理の結果を待って次へ進む

1. `await fetch(...)` でレスポンスを待つ
2. `await response.json()` で本文を読み取り、JSONを解析する
3. `response.ok` を確認する

待っている間も、ブラウザ全体が固まるわけではありません。

HTTPエラーの扱い

`fetch()` は、HTTP 400や500でも例外になりません。

```
if (!response.ok) {
  // サーバーが返したエラーの説明を表示する。
  let message = '入力内容を確認してください。';
  if (typeof data.detail === 'string') {
    message = data.detail;
  }
  throw new Error(message);
}
```

`HTTPException` の `detail` が `data.detail` として届きます。
通信自体の失敗と、サーバーが返したエラーを区別します。

失敗したら履歴を戻す

```
} catch (error) {
  // 送れなかった質問は履歴から外し、入力欄に戻す。
  history.pop();
  showMessages();
  questionInput.value = question;
  statusText.textContent = error.message;
} finally {
  sendButton.disabled = false;
  resetButton.disabled = false;
}
```

状態	履歴	入力欄	ボタン
成功	回答を足す	空のまま	有効に戻す
失敗	質問を外す	質問を戻す	有効に戻す

実習1：完成版を動かして観察

`sample-app/chat-app/` を動かしたまま、開発者ツールのNetworkタブを開きます。

1. 「私の名前は佐藤です。」を送る
2. 一覧から `/api/chat` を選ぶ
3. 「私の名前は？」を送り、もう一度 `/api/chat` を選ぶ

欄	確認する値
Method	POST
Request Payload	`history` に会話が全部入っている
Status	200
Response	`answer`、`input_tokens`、`output_tokens`

Payload が長くなる

送信	Payload の `history`	入力トークン数
1回目	1件	少ない
2回目	3件	増える
3回目	5件	さらに増える

2回目の Payload には、1回目の質問と回答も入っています。
外部LLMへの通信はサーバー側なので、Networkタブには表示されません。
一覧が空なら、送信前からタブを開いて再度試します。

実習2：app.js の送信を埋める

完成版のサーバーを `Ctrl+C` で止め、穴あき版を起動します。

```
cd ../../session03/exercise
python main.py
```

手順	すること
1	埋める前に送信する。ステータス欄にTODOの文が出て、質問が入力欄に戻る
2	`static/app.js` の `sendQuestion()` で `TODO` を探す
3	`fetch(...)` と `response.json()` のコメントを外す
4	`throw new Error('TODO: ...')` の行を消す
5	ページを再読み込みして送信する

実習2の確認：501を観察

埋めた後に送信すると、サーバーにはまだ穴があるので 501 が返ります。

観察	分かること
Networkタブに `/api/chat` が出る	送信は届いた
Status が 501	`main.py` の仮の `HTTPException`
ステータス欄に `TODO: main.py の chat() で…`	`detail` が `data.detail` として画面に届いた
質問が入力欄に戻る	`catch` の `history.pop()` が動いた

サーバーが返した `detail` が、画面に届く経路を確認できます。

実習3：main.py の確認を埋める

手順	すること
1	`main.py` の `chat()` で `TODO` を探す
2	3つの `if` と `raise HTTPException(status_code=400, …)` のコメントを外す
3	`raise HTTPException(status_code=501, …)` の行を消す
4	サーバーを `Ctrl+C` で止めて `python main.py` で起動し直す
5	「私の名前は佐藤です。」の次に「私の名前は？」と送る

`main.py` を直した後は、サーバーの再起動が必要です。
`app.js` を直した後は、ページの再読み込みだけで反映されます。

実習4：/docs から異常な入力

ポート8000番の `/docs` を開き、POST `/api/chat` の「Try it out」を使います。

送る `history`	期待する結果
`[]`	422
`role` が `system` の1件	422
`content` が `""` の1件	422
最後が `assistant`	400
501文字の `user`	400
`user` と `assistant` を交互に21件	400

正しい履歴を送ると、実際にLLM APIを呼びます。

失敗した場所の切り分け

観察した状態	まず確認すること
画面が開かない	サーバー起動、ポート8000番、`app.mount` の位置
TODOの文が出て `/api/chat` が無い	`app.js` の穴埋めと再読み込み
501	`main.py` の仮の `raise`
422	送ったJSONの形、`role` と `content`
400	`chat()` の3つの確認、質問の回数
502・504・429	キー・モデル・利用枠、ネットワーク

相談時は、Status と ステータス欄の文を伝えます。キーは共有しません。

確認問題

1. `fetch('/api/chat')` は誰に送信している？
2. APIキーが必要なのは、2つの通信のどちら？
3. 画面の `MAX_TURNS` で止めているのに、サーバーでも回数を確かめるのはなぜ？
4. 422と400は、それぞれどこで止まった印？

提出物と次回

`session03/exercise/` の `static/app.js` と `main.py` をコミットし、プッシュします。

```
cd ../..  # リポジトリのルートへ戻る
git add session03/exercise
git commit -m "第3回: チャットアプリの送信と検証"
git push
```

GitHubで `session03/exercise` を開き、そのURLをフォームから提出します。
APIキーや、APIの生の例外全文は提出しません。

次回は、LLMへ授業資料を渡し、回答の根拠を確認します。